

UNIT III

COLLECTIONS

LIST

Definition

- List is a structure data type of python that is **collection of sequence of values belonging of any type**. List can store different types of values.
- List elements should be given inside the square bracket **[]**.
- Lists are **mutable**: (i.e.) modifiable: we can **change the elements of list at any time** you want.

CREATING A LIST

To create a list, add a number of expressions, separated by comma within a square bracket.

Syntax: `var_name=[element1,element2,.....element n]`

Example:

`List1=[]` `#empty list`

`List2=[1,2,3]` `#list with values of same type` `List3=[1,`
`“a”,2,”s”,3,”red”,4]`

`#list with values of different types`

Example for creating list with for loop

```
list=[ ]
```

```
print("Enter 10 no:") for i in
```

```
range (10):
```

```
    list.append(eval(input()))
```

LIST VS STRING

Lists and strings have many similarities and differences.

Similarities between lists and strings

List and string use same function as

- length: `len(variable_name)`
- indexing: `L[i]`
- slicing: `L[initial:end:step value]`
- membership operator: `in` & `not in`
- concatenation and replication operators: `+` - concatenate, `*` - from replication]
- Accessing individual elements: `L[index]`

Differences between list and string

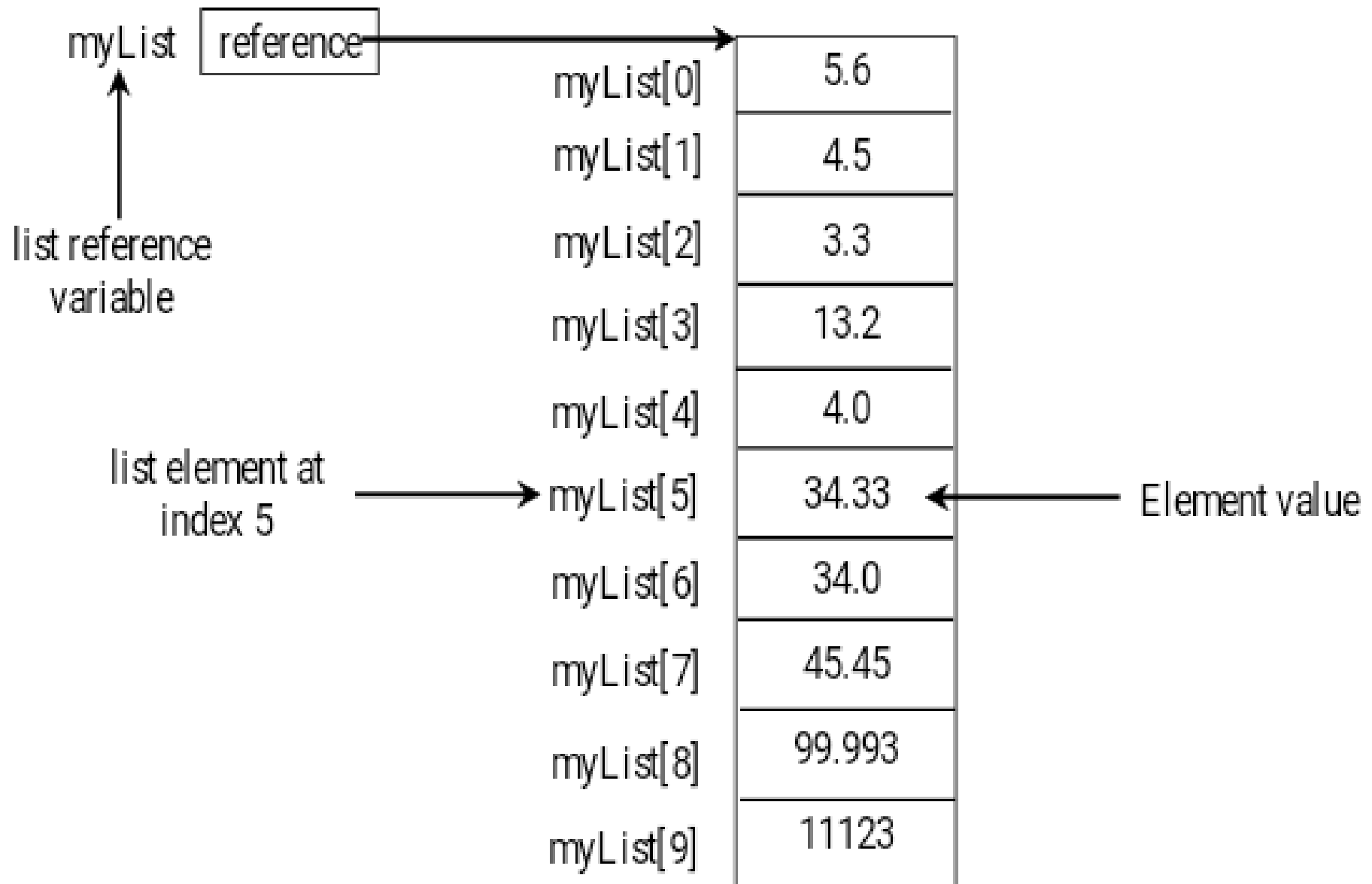
Mutability: String is immutable but list is mutable.

Storage: List is stored in memory exactly like string, except that the objects are larger than others.

Index operators []: An element in the list can be accessed through index operator.

Syntax: list_variable[index]

myList = [5. 6, 4. 5, 3. 3, 13. 2, 4. 0, 34. 33, 34. 0, 45. 45, 99. 993, 11123]



```
my_list=[1,2,3.4,5,6.789,"python",1+2i,"x"]
```

Reverse indexing	-8	-7	-6	-5	-4	-3	-2	-1
List values	1	2	3.4	5	6.789	python	1+2i	x
Forward indexing	0	1	2	3	4	5	6	7

for printing value in the list

```
print(my_list[0])
```

Output: 1

TRAVERSING A LIST/LIST LOOPING

Traversing a list means accessing and processing each of the elements in a list(visiting all the elements once). For loop is mainly used to traverse a list.

Syntax:

```
for variable_name in list_name: Process each item  
    Print or operations
```

File Edit Format Run Options Window Help

```
my_list = [ "p","y","t","h","o","n" ]  
print("traversing list with for loop")  
for letter in my_list:  
    print(letter,end=",")
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

```
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:  
on win32  
Type "copyright", "credits" or "license()" for  
>>>  
===== RESTART: C:\Users\VISWANATH\Desktop  
traversing list with for loop  
p,y,t,h,o,n,  
>>> |
```

- **Joining or concatenation of list**

The '+' operator is used to concatenate or join two list into single list.

Syntax: var_name= list1 + list 2

```
list1 = [ "welcome:", "to" ]
list2 = [ "python" , "programming" ]
list3 = list1 + list2
print(" LIST1 + LIST2 CONCATENATED AS:")
print(list3)
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:02:03) on win32

Type "copyright", "credits" or "license()" for more
>>>

===== RESTART: C:\Users\VISWANATH\Desktop>>>

```
LIST1 + LIST2 CONCATENATED AS:
['welcome:', 'to', 'python', 'programming']
>>>
```

- **Repeating or replicating list**

Like string, we can use “*” operator to replicate a list to specified number of times.

Syntax: var_name=list_name * no_of_times

```
list1 = [1,2,3]
list1replication = list1 * 3
print("list1 is replicated 3 times")
print(list1replication)
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:04:44) on win32

Type "copyright", "credits" or "license()" for more
>>>

===== RESTART: C:\Users\VISWANATH\De

list1 is replicated 3 times

[1, 2, 3, 1, 2, 3, 1, 2, 3]

LIST SLICE

List slice is same of string slice. From the list slice we can **extract sub part of list** based on user requirements. We can use index values to create a sublist from list.

Syntax:

var_name=list_name[start value : end value: increment/decrement step value]

Here starting and ending value may be omitted or optional. Default value for start is zero and last is last index. Default step value is: +1 or simply 1.

```
list1 = [0,1,2,3,4,5,6,7,8,9,10]
list1_slice = list1[0 : 5 : 1 ]
print("FULL LIST:",list1)
print("SLICED LIST VALUES:",list1_slice)
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:46) on win32

Type "copyright", "credits" or "license()" for more
>>>

===== RESTART: C:\Users\VISWANATH\Desktop>>>

FULL LIST: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

SLICED LIST VALUES: [0, 1, 2, 3, 4]

>>> |

```
list1 = [0,1,2,3,4,5,6,7,8,9,10]
print (list1[ : : ])
print (list1[0 : :])
print (list1[ :-1 :])
print (list1[: :2])
print (list1[ :2 :])
print (list1[3: :])
print (list1[-4 : -2:+1])
# List[-4:-2:-1]=[-4+len(list) : -2+len(list): -1]
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52)
on win32

Type "copyright", "credits" or "license()" for more
>>>

===== RESTART: C:\Users\VISWANATH\Desktop\p

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

[0, 2, 4, 6, 8, 10]

[0, 1]

[3, 4, 5, 6, 7, 8, 9, 10]

[7, 8]

>>> |

LIST METHODS AND FUNCTIONS

Python list provides many built-in function and methods for list manipulation. These can be applied to list as per the syntax below:

Syntax: <list object>. <methodname>()

The list has various list class methods. They are

- **append()**
- **count()**
- **extend ()**
- **index ()**
- **insert ()**
- **pop ()**
- **remove ()**
- **reverse ()**
- **sort ()**
- **del ()**
- **len ()**
- **min ()**
- **max ()**
- **sum ()**

Consider list 1 = [2, 3, 4, 1, 32, 4]

list 2 = [99, 54]

append:

The append() method adds an new elements to the end of list.

Syntax: list _name .append (new elements)

Example: (list1 .append (19))

Output: [2, 3, 4, 1, 32, 4, 19]

count:

The count returns a integer value which is the number of times a particular elements x appear in the list.

Syntax: list _name. count (object) **Example:**

print (list1.count (4)) **Output:** 2

extend:

This appends all the elements in the list 2 at end of list 1.

Syntax: list1.extend (list 2) // list or any one value **Example:** print (list1.extend (list2))

Output: [2, 3, 4, 1, 32, 4, 19, 99, 54]

index:

This returns the index of the first occurrence of element x in the list.

Syntax:list_name.index (element) **Example:** print (list1.index (4)) **Output:** 2

insert:

This inserts an element x at any given index. Note that first element in the list has index 0.

Syntax: **list_name.insert (index value, the element to be inserted)**

Ex: print (list1.insert (1,25)) output:

[2,**25**,3,4,1,32,4,19,99,54]

pop:

This **removes an element at given position and returns it**. The index parameter is optional. If we specified the index then pop will remove particular indexed value. If index is not specified pop removes the last element in list.

Syntax: list_name.pop (index) #index is optional.

Example 1: print (list 1. pop (2) output: [2, 25, 4, 1, 32, 4, 19, 99, 54]

Example 2: print (list 1. pop()) output: [2,25,4,1,32,4,19,99]

remove:

This removes the first occurrence of element x in the list.

Syntax: list_name.remove (element we want to remove)

Example: list1.remove (32)

Output: [2, 25, 4, 1, 4, 19, 99]

reverse:

It reverses the elements in the list.

Syntax: list_name.reverse () **Example:**

list1.reverse () **Output:** 99, 19,4,1,4,25,2

sort:

It sorts the elements in the list in **ascending order**.

Syntax: `list_name.sort ()`

Example: `list1.sort ()`

Output: `[1, 2, 4, 4, 19, 25, 99]`

To sort the list in **descending order**:

Syntax: `list _name. sort (reverse = true)`

Example: `list1 = [0, 100, 90, 151, 1024, 1]
print (list 1.sort (reverse = True))`

Output: `[1024, 151, 100, 90, 1, 0]`

del:

This deletes a particular element in the list based on index or value.

Syntax for deleting with index: `del list _name [index value]`

Syntax for deleting with range: `del list_name [start value :end value: step value]`

If we don't know the index value of the character then use:

`list_name.remove` (“character/ value” we want to
remove)

Example:

```
list1 = [1,2,3,4,5,6,7,8,9,10,11,12]
```

```
del list1 [7]                      Output: [1,2,3,4,5,6,7,9,10,11,12]
```

```
del list1 [8:10]                      Output: [1,2,3,4,5,6,7,9,12]
```

To delete the entire list element:

del list_name #without index delete all values list. **del list1**

Output: []

len:

Len function is mainly used find out the number of elements in the list.

Syntax: `val_name = len (list _name)`

Example: `list 1 = [0, 1, 2, 3, 4, 5] print (len (list 1) )`

Output: 6

min:

Min returns the minimum value that present in the list.

It is an inbuilt function.

Syntax: `min (list _name)`

Example: `print (min (list1))` // [3,10,5,6] //['a',5,3.3,9]

Output: 0

max:

Max returns the maximum value in the list.

Syntax: max (list _name)

Example: print (max (list 1))

Output: 5

sum:

Sum returns the total value by adding all the elements in the list.

Syntax:sum(list_name)

Example:print(sum(list1)) **Output:** 15

```
list1 = [ 2, 3, 4, 1, 32, 4 ]
list2 = [ 99, 54 ]

list1.append(19)
print("LIST1 APPENED WITH 19:",list1)

print("count of 4 present in list:",list1.count(4))

list1.extend(list2)
print("Extend list1 with list2:",list1)

print("index number of the value 4 is:",list1.index(4))

list1.insert(1,25)
print("insert element @ particular place:",list1)

list1.pop(2)
print("POPING items based on value:",list1)

list1.reverse()
print("REVERSING THE LIST:",list1)

list1.sort()
print("sorting the list in ascending order:",list1)

list1.sort(reverse = True)
print("sorting the list in descending order:",list1)

print("MINIMUM VALUE IN THE LIST:",min(list1))

print("MAXIMUM VALUE IN THE LIST:",max(list1))
```

OUTPUT

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit
on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =====
LIST1 APPENED WITH 19: [2, 3, 4, 1, 32, 4, 19]
count of 4 present in list: 2
Extend list1 with list2: [2, 3, 4, 1, 32, 4, 19, 99, 54]
index number of the value 4 is: 2
insert element @ particular place: [2, 25, 3, 4, 1, 32, 4, 19, 99, 54]
POPING items based on value: [2, 25, 4, 1, 32, 4, 19, 99, 54]
REVERSING THE LIST: [54, 99, 19, 4, 32, 1, 4, 25, 2]
sorting the list in ascending order: [1, 2, 4, 4, 19, 25, 32, 54, 99]
sorting the list in descending order: [99, 54, 32, 25, 19, 4, 4, 2, 1]
MINIMUM VALUE IN THE LIST: 1
MAXIMUM VALUE IN THE LIST: 99
>>> |
```

LIST MUTABILITY

Unlike string, lists are mutable i.e., we can add / delete / replace any value in list at any time.

Syntax: list_name [index] = value

List loop is the same as list traversing.

```
list1 = [ 2, 3, 4, 1, 32, 4 ]
print("old list vales:",list1)
print("now we are replacing vlaue 1 with 100:")
list1[3] = 100
print("new list values:",list1)
```

Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52)
on win32

Type "copyright", "credits" or "license()" for more
>>>

===== RESTART: C:\Users\VISWANATH\Desktop>

old list vales: [2, 3, 4, 1, 32, 4]

now we are replacing vlaue 1 with 100:

new list values: [2, 3, 4, 100, 32, 4]

Aliasing and Cloning the list

Aliasing:

Since variables refer to objects, if we assign one variable to another, both variables refer to the same object

Cloning:

- If we want to modify a list and also keep a copy of the original, we need to be able to make a copy of the list itself, not just the reference. This process is sometimes called **cloning**, to avoid the ambiguity of the word copy.
- The easiest way to clone a list is to use the slice operator.

Aliasing

```
List1=[1,2,3,4,5]
```

```
List2=List1      #aliasing
```

```
print("List1=",List1)
print("List2=",List2)
List2[2]=50
print("List1=",List1)
print("List2=",List2)
```

Output:

```
List1= [1, 2, 3, 4, 5]
List2= [1, 2, 3, 4, 5]
List1= [1, 2, 50, 4, 5]
List2= [1, 2, 50, 4, 5]
```

Cloning

```
List1=[1,2,3,4,5]
```

```
List2=List1[:] #cloning
```

```
print("List1=",List1)
```

```
print("List2=",List2)  
List2[2]=50
```

```
print("List1=",List1)
```

```
print("List2=",List2)
```

Output:

```
List1= [1, 2, 3, 4, 5]
```

```
List2= [1, 2, 3, 4, 5]
```

```
List1= [1, 2, 3, 4, 5]
```

```
List2= [1, 2, 50, 4, 5]
```

```
list1 = [1,2,3]
list2 = list1
print("list1 values:",list1)
print("list2 values:",list2)
list1[0] = 100
print("now updated first value in the list with 100")
print("New list1 values:",list1)
print("New list2 values:",list2)
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

```
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [
  on win32
Type "copyright", "credits" or "license()" for more in
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pys
list1 values: [1, 2, 3]
list2 values: [1, 2, 3]
now updated first value in the list with 100
New list1 values: [100, 2, 3]
New list2 values: [100, 2, 3]
>>> |
```

The IN and NOT IN operator:

- We can determine whether an element is present in the list by using in or not in operator.
- IN operators returns true if particular element is present, else it returns false.
- Not IN operator returns true if particular element is not present, else it returns false.

File Edit Format Run Options Window Help

```
list1 = [1,2,3]
print("list contents:",list1)
print("is 1 is present in list1:", 1 in list1)
print("is 100 is present in list1:",100 not in list1)
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [Python]
on win32

Type "copyright", "credits" or "license()" for more in:

>>>

===== RESTART: C:\Users\VISWANATH\Desktop\pys

list contents: [1, 2, 3]

is 1 is present in list1: True

is 100 is present in list1: True

>>> |

The conditional operators (==,!=,>,>=,<,<=) are also used with lists to check whether two lists are equal, not equal etc.

```
list1 = [ "green", "red", "blue" ]
list2 = [ "red", "blue", "green" ]
print("is list1 == list2:",list1 == list2)
print("is list1 != list2:",list1 != list2)
print("is list1 <= list2:",list1 <= list2)
print("is list1 < list2:",list1 < list2)
print("is list1 >= list2:",list1 >= list2)
print("is list1 > list2:",list1 > list2)
```

Python 3.6.1

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 1
on win32

Type "copyright", "credits" or "license()" i
>>>

===== RESTART: C:\Users\VISWANATH\I

is list1 == list2: False

is list1 != list2: True

is list1 <= list2: True

is list1 < list2: True

is list1 >= list2: False

is list1 > list2: False

PASSING LIST TO FUNCTION

- While passing a list to a function, the elements of list may change due to function operation because list is mutable.
- In the above code, after m() function is invoked x value is 1 because number is immutable. But y[0] is changed to 5555 as list is mutable.

```
def m(number, numbers):  
    number = 1001 # Assign a new value to number  
    numbers[0] = 5555 # Assign a new value to numbers[0]  
  
x = 1 # x represents an int value  
y = [1, 2, 3] # y represents a list  
m(x, y) # Invoke f with arguments x and y  
print("x is", x)  
print("y[0] is", y[0])
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC
on win32

Type "copyright", "credits" or "license()" for more infor
>>>

===== RESTART: C:\Users\VISWANATH\Desktop\pysamp

x is 1

y[0] is 5555

RETURN A LIST FROM FUNCTION

- In python a list can also be returned from a function. When a function returns a list, the list reference value is returned.

```
def reverse(list2):  
    result=[]  
    for element in list2:  
        result.insert(0,element)  
    return result  
  
list1=[1,2,3,4,5,6]  
print("ORIGINAL LIST:",list1)  
print("USING FUNCTION REVERSING THE LIST")  
print("AFTER REVERSING WE ARE RETURNING LIST FROM FUNCTION")  
revlist1=reverse(list1)  
print("REVERSED LIST USING FUNCTION:",revlist1)
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1
on win32

Type "copyright", "credits" or "license()" for more informati
>>>

===== RESTART: C:\Users\VISWANATH\Desktop\pysample.p

ORIGINAL LIST: [1, 2, 3, 4, 5, 6]

USING FUNCTION REVERSING THE LIST

AFTER REVERSING WE ARE RETURNING LIST FROM FUNCTION

REVERSED LIST USING FUNCTION: [6, 5, 4, 3, 2, 1]

>>> |

SPLITTING A STRING INTO A LIST

- The str class contains the split method, which is useful for splitting items in a string into a list.

```
string1="python program welcome you all"
list_of_string1=string1.split()
print(list_of_string1)
string2="09/09/2017"
list_of_string2=string2.split("/")
print(list_of_string2)
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54
on win32)

Type "copyright", "credits" or "license()" for more
>>>

===== RESTART: C:\Users\VISWANATH\Desktop>

```
['python', 'program', 'welcome', 'you', 'all']
['09', '09', '2017']
```

```
>>> |
```

TUPLES

Definition: Tuples are like list, but their elements are fixed. Once a tuple is created, you cannot add /delete/reorder/replace an element in tuple. Tuple is immutable in nature.

CREATING TUPLES:

Tuples can be defined as elements separated by comma within parenthesis. **Syntax:**

```
var_name=(element1,element2,...elementn)
```

Example: tuple_values=(1,2,3,4,5,6,7,8,9,10)

Creating empty tuple :

```
tuple1=()
```

Creating tuple with one element:

```
tuple2=(1)
```

If you create a tuple with single element python considers it as a value assigned and tuple2 as a variable name

In order to avoid that, include “,” operator next to that element so that python considers it as a tuple (i.e) tuple2=(1,).

Creating tuple from string:

```
#creating tuples from string
string1 = "hello"
print("origianl string:",string1)
tuple1 = tuple(string1)
print("string converted to tuples:",tuple1)
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [M
on win32

Type "copyright", "credits" or "license()" for more inf
>>>

```
===== RESTART: C:\Users\VISWANATH\Desktop\pysa
origianl string: hello
string converted to tuples: ('h', 'e', 'l', 'l', 'o')
>>>
```

Creating tuple from list:

```
#creating tuples from list
list1 = ["W","E","L","C","O","M","E","to","python"]
print("original list:",list1)
tuple2 = tuple(list1)
print("list converted to tuples:",tuple2)
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) |
on win32

Type "copyright", "credits" or "license()" for more in
>>>

```
===== RESTART: C:\Users\VISWANATH\Desktop\py:
original list: ['W', 'E', 'L', 'C', 'O', 'M', 'E', 'to']
list converted to tuples: ('W', 'E', 'L', 'C', 'O', 'M', 'E', 'to')
>>> |
```

Creating tuple from keyboard input:

```
#CREATING TUPLE FROM KEYBOARD INPUT
```

```
#Method:1
```

```
tuple1=tuple(input("Enter tuple elements method1:"))  
print(tuple1)
```

```
#Method: 2
```

```
tuple2=eval(input("Enter tuple elements method2:"))  
print(tuple2)
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [IPython]
on win32

Type "copyright", "credits" or "license()" for more in
>>>

===== RESTART: C:\Users\VISWANATH\Desktop\pys

Enter tuple elements method1:123456789

('1', '2', '3', '4', '5', '6', '7', '8', '9')

Enter tuple elements method2:1,"a",2,"bcd",3.14,"qwerty"

(1, 'a', 2, 'bcd', 3.14, 'qwerty')

>>> |

TUPLE vs. LIST

Similarities:

Length fun is used on both tuple and list are same Indexing and slicing are same on both tuples and list Both concatenation and exponentiation operation are possible

Both in and not in operation allowed in tuples and list

Difference:

Tuples are immutable and lists are mutable

TUPLE OPERATIONS

Traversing
Joining the tuple

Replicating the tuple
Slicing the tuple
Unpacking the tuple

Traversing a tuple:

Using for loop we can easily traverse the individual elements in a tuple.

Syntax:

```
for var_name in range(0,len(tuple_name),1): process each item  
    statement
```

```
my_tuple1 = ("p","u","r","e")
for element in range(0,len(my_tuple1),1):
    print("element",element,"is:",my_tuple1[element])
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52)
on win32

Type "copyright", "credits" or "license()" for more in
>>>

===== RESTART: C:\Users\VISWANATH\Desktop\py:

element 0 is: p

element 1 is: u

element 2 is: r

element 3 is: e

>>> |

Joining the tuple:

The “+” operator is used to concatenate the two tuples

Syntax: tuple3=tuple1+tuple2

```
File Edit Format View Options Window Help
tuple1 = (1,2,3)
tuple2 = ("a","b","c")
print("tuple1 is:",tuple1)
print("tuple2 is:",tuple2)
tuple3 = tuple1 + tuple2
print("tuple1+tuple2 is:",tuple3)
```



Python 3.6.1

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 1
on win32

Type "copyright", "credits" or "license()" f
>>>

===== RESTART: C:\Users\VISWANATH\D

tuple1 is: (1, 2, 3)

tuple2 is: ('a', 'b', 'c')

tuple1+tuple2 is: (1, 2, 3, 'a', 'b', 'c')

>>> |

Repeating or replicating tuples:

Like string and list, we can use * operator to replicate tuple to a specified number of times

Syntax: tuple_replicate_name=tuple_name * number of times to be replicated

File Edit Format Run Options Window Help

```
tuple1 = (1,2,3)
print("original tuple is:",tuple1)
tuple1_replicate = tuple1 * 3
print("replicated tuple is:",tuple1_replicate)
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

```
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:
on win32
```

```
Type "copyright", "credits" or "license()" for mc
>>>
```

```
===== RESTART: C:\Users\VISWANATH\Desktop
```

```
original tuple is: (1, 2, 3)
```

```
replicated tuple is: (1, 2, 3, 1, 2, 3, 1, 2, 3)
```

Slicing of tuple:

Like string and list slice, tuple slice is same with based on user requirements to extract sub tuple.

Syntax: var _name =tuple_name[startvalue:endvalue:stepvalue]

```
File Edit Format Run Options Window Help
tuple1 = [0,1,2,3,4,5,6,7,8,9,10]
print ("tuple1[ : : ]      :",tuple1[ : : ])
print ("tuple1[0 : : ]     :",tuple1[0 : : ])
print ("tuple1[ : -1 : ]    :",tuple1[ : -1 : ])
print ("tuple1[: : 2]       :",tuple1[: : 2])
print ("tuple1[ : 2 : ]     :",tuple1[ : 2 : ])
print ("tuple1[3: : ]       :",tuple1[3: : ])
print ("tuple1[-4 : -2 : +1]:",tuple1[-4 : -2 : +1])
```

 Python 3.6.1 Shell

```
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52)
on win32
Type "copyright", "credits" or "license()" for more
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\p
tuple1[ : : ]      : [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
tuple1[0 : : ]     : [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
tuple1[ : -1 : ]    : [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
tuple1[: : 2]       : [0, 2, 4, 6, 8, 10]
tuple1[ : 2 : ]     : [0, 1]
tuple1[3: : ]       : [3, 4, 5, 6, 7, 8, 9, 10]
tuple1[-4 : -2 : +1]: [7, 8]
>>> |
```

Here starting and ending value may be omitted or optional.

Default :

Start value is zero and last value is last index. (Here it is 11)

Step value is: +1 or simply 1

Unpacking the tuple

Creating a tuple from set of values is called packing and its reverse i.e., creating individual elements from tuple is called unpacking

Syntax: <var_1>, <var_2>, <var_3>.....=tuple_name

Where the elements (number of the variables) in the left side must match with number of elements in the tuple

```
t=(1,2,"A","B")
print("original tuple:",t)
print("length of tuple:",len(t))
w,x,y,z=t
print("unpacked first element from tuple si:",w)
print("unpacked second element from tuple si:",x)
print("unpacked third element from tuple si:",y)
print("unpacked fourth element from tuple si:",z)
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:5
on win32

Type "copyright", "credits" or "license()" for mor
>>>

===== RESTART: C:\Users\VISWANATH\Desktop

original tuple: (1, 2, 'A', 'B')

length of tuple: 4

unpacked first element from tuple si: 1

unpacked second element from tuple si: 2

unpacked third element from tuple si: A

unpacked fourth element from tuple si: B

>>> |

TUPLES FUNCTION AND METHODS

Tuple mainly contains 5 built-in functions. They are: len()

min()

max()

cmp()

sum()

len() method: Returns the length of the tuple.

Syntax:len(tuple_name)

min() method:Find the minimum element from the tuple.

Syntax:min(tuple_name)

max() method: Find out maximum value element from tuple

Syntax: max(tuple_name)

cmp() method: The cmp() method used to compare two tuples based on their elements.

Syntax: cmp(tuple1,tuple2)

Returns the value

0 (zero) if both tuples are same or equal

-1 if tuple1 < tuple2

+1 if tuple1 > tuple2

sum() method: The sum() method is used to sum or add all the elements in the tuple if its integer values and float values

Syntax: sum(tuple_name)

```
File Edit Format Run Options Window Help
tuple1 = (0,1,2,3,4,5,6,7,8,9,10)
tuple2 = (0,1,2,3,4)
print("TUPLE1 IS:",tuple1)
print("LENGTH OF TUPLE1 IS:",len(tuple1))
print("MINIMUM VALUE IN TUPLE1:",min(tuple1))
print("MAXIMUM VALUE IN TUPLE2:",max(tuple1))
print("SUM OF TUPLE1 is:",sum(tuple1))
print("TUPLE2 IS:",tuple2)
print("is tuple1 equal to tuple2:",cmp(tuple1,tuple2))
```



Python 3.6.1 Shell

```
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [M
on win32
Type "copyright", "credits" or "license()" for more inf
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pysa:
TUPLE1 IS: (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
LENGTH OF TUPLE1 IS: 11
MINIMUM VALUE IN TUPLE1: 0
MAXIMUM VALUE IN TUPLE2: 10
SUM OF TUPLE1 is: 55
TUPLE2 IS: (0, 1, 2, 3, 4)
```

TUPLES AS RETURN VALUES

- Functions usually return a value but if the value is a tuple then function can return multiple values.
- **Example:** If you want to divide two integers and compute the quotient and remainder. It is inefficient to compute x/n and $x\%n$. It is better to compute them both at same time
- The built in function `divmod` takes two arguments or integer values and return two values: quotient and remainder.

```
#method : 1 :  
print("QUOTIENT & REMAINDER IS:")  
tuple1=divmod(7,3)  
print(tuple1)  
  
#method : 2:  
quotient,remainder=divmod(7,3)  
print("quotient = ",quotient)  
print("remainder =",remainder)
```



File Edit Shell Debug Options Window

Python 3.6.1 (v3.6.1:69c0db5, Mar
on win32

Type "copyright", "credits" or "li
>>>

===== RESTART: C:\Users\W

QUOTIENT & REMAINDER IS:

(2, 1)

quotient = 2

remainder = 1

>>> |

Example

```
def fun(tuple2):  
    print("tuple2=",tuple2)  
    tuple3=(3,4,5)  
    return tuple3    #returning tuple to calling function
```

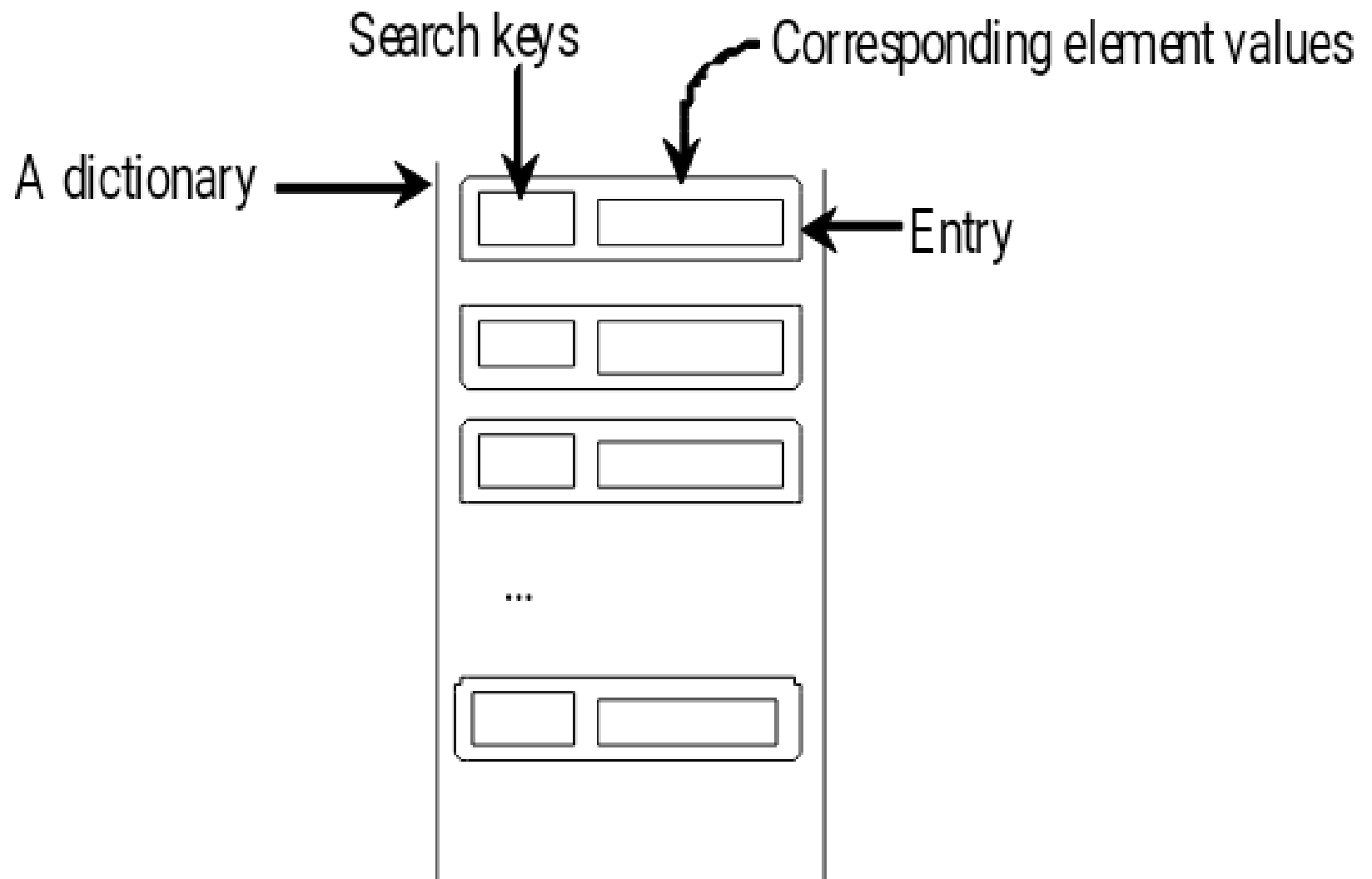
```
tuple1=(5,10,15,20)  
#tuple4=fun(tuple1)  #passing tuple to  
function #print("tuple 4=",tuple4)
```

```
x,y,z=fun(tuple1)    #x,y,z are used to unpack  
the tuple print(x)  
print(y)  
print(z)
```

DICTIONARIES

Definition:

A dictionary is a container object that stores a collection of **key: value** pairs. It ensures **fast retrieval, deletion and updating of value by using a key.** **Dictionary is mutable and unordered collection.** Here key is like an index operator i.e., key must be unique within dictionary. It is also called as a **map**, where each key is mapped or assigned with value of any data type.



Creating a dictionary

- You can create a dictionary by enclosing the item inside a pair of curly braces {}. Each item consists of a key, followed by colon followed by value. Each item may be separated with commas.
- **Syntax:** <dictionary_name>={key1:value, key2:value,...keyn:value}
- **Example:** Indian_player_dictionary={1: “dhoni”, 2: “kholi”, 3: “shewag”}
- To create empty dictionary: <dict_name>={}

Accessing elements of dictionary

- In dictionary elements are accessed by their keys only.
- **Syntax:** `var_name=dictionary_name[key]`
- An attempt to access a key that doesn't exist in the dictionary will cause an error.

```
#creating dictionary
indian_player_dictionary={1:"dhoni",2:"kholi",3:"shewag",4:"gambhir",5:"virat",6:"kohli",7:"sachin",8:"sehwag",9:"tendulkar",10:"dravid"}

#accessing the elements
india_select=indian_player_dictionary[1]
print("selected player key 1 is:",india_select)

india_select=indian_player_dictionary[10]
print("selected player key 10 is:",india_select)
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC
on win32

Type "copyright", "credits" or "license()" for more infor
>>>

===== RESTART: C:\Users\VISWANATH\Desktop\pysamp.

selected player key 1 is: dhoni

Traceback (most recent call last):

File "C:\Users\VISWANATH\Desktop\pysample.py", line 8, :
india_select=indian_player_dictionary[10]

KeyError: 10

>>>

To access all the keys and value simultaneously:

<dictionary_name>.keys()

```
indian_player_dictionary={1:"dhoni",2:"kholi",3:"shewag",4:"ashwin"}  
print("ALL KEYS IN DICT IS:",indian_player_dictionary.keys())  
print("ALL VALUES IN DICT IS:",indian_player_dictionary.values())
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit (Intel) on win32

Type "copyright", "credits" or "license()" for more information.

>>>

===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =====

ALL KEYS IN DICT IS: dict_keys([1, 2, 3, 4])

ALL VALUES IN DICT IS: dict_values(['dhoni', 'kholi', 'shewag', 'ashwin'])

>>>

Characteristics of dictionary

Like list, dictionary is also mutable (we can add/delete/update values). In dictionary keys are immutable and values are mutable

Dictionary is unordered set of key: value pairs.

Unlike list, string, tuples, dictionary is not in sequence memory location.

Dictionary is indexed with keys, where each key must be unique.

Dictionary operations

Traversing

Adding an element Updating
the elements Deleting the
elements

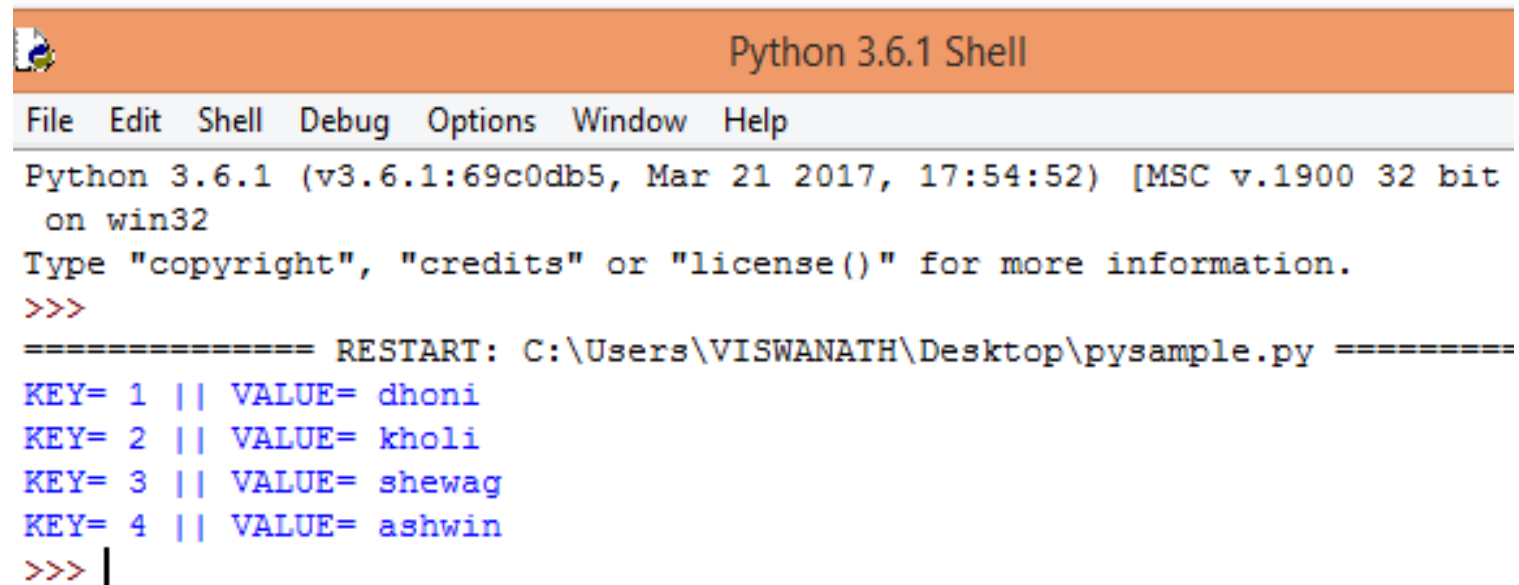
Checking for presence of key

Traversing the dictionary: For loop is mainly used to traverse over the items of the dictionary.

Syntax:

for var_name in dictionary_name

```
File Edit Format View Options Window Help
indian_player_dictionary={1:"dhoni",2:"kholi",3:"shewag",4:"ashwin"}
for element in indian_player_dictionary:
    print("KEY=",element,"|| VALUE=",indian_player_dictionary[element])
```

A screenshot of a Python 3.6.1 Shell window. The title bar is orange and says "Python 3.6.1 Shell". Below it is a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main text area shows the output of a Python script. It starts with "Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit on win32]" and "Type 'copyright', 'credits' or 'license()' for more information." followed by ">>>". Then a separator line "===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =====" appears. Below this, the output of the for loop is shown: "KEY= 1 || VALUE= dhoni", "KEY= 2 || VALUE= kholi", "KEY= 3 || VALUE= shewag", and "KEY= 4 || VALUE= ashwin". The prompt ">>>|" is at the bottom.

```
Python 3.6.1 Shell
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit
on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =====
KEY= 1 || VALUE= dhoni
KEY= 2 || VALUE= kholi
KEY= 3 || VALUE= shewag
KEY= 4 || VALUE= ashwin
>>> |
```

e: Processing each item tree.

Adding element to dictionary: We can add new element (key: value) to dictionary using assignment operator as follow.

Syntax:

dictionary_name [new_key]=v

```
indian_player_dictionary={1:"dhoni",2:"kholi",3:"shewag",4:"ashwin"}
indian_player_dictionary[5] = "ganguly"
for element in indian_player_dictionary:
    print("KEY=",element,"|| VALUE=",indian_player_dictionary[element])
```



```
Python 3.6.1 Shell
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit
on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =====
KEY= 1 || VALUE= dhoni
KEY= 2 || VALUE= kholi
KEY= 3 || VALUE= shewag
KEY= 4 || VALUE= ashwin
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =====
KEY= 1 || VALUE= dhoni
KEY= 2 || VALUE= kholi
KEY= 3 || VALUE= shewag
KEY= 4 || VALUE= ashwin
KEY= 5 || VALUE= ganguly
>>> |
```

alue

Make sure that, when adding a new key, the value of the key must not present in the existing dictionary. If the key is already present, then it will update the old value of the key with the new value you are assigning.

```
indian_player_dictionary={1:"dhoni",2:"kholi",3:"shewag",4:"ashwin"}
print("dictionary before delete:",indian_player_dictionary)
del indian_player_dictionary[4]
print("dictionary after delete:",indian_player_dictionary)
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit (Intel) on win32]

Type "copyright", "credits" or "license()" for more information.

>>>

===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =====

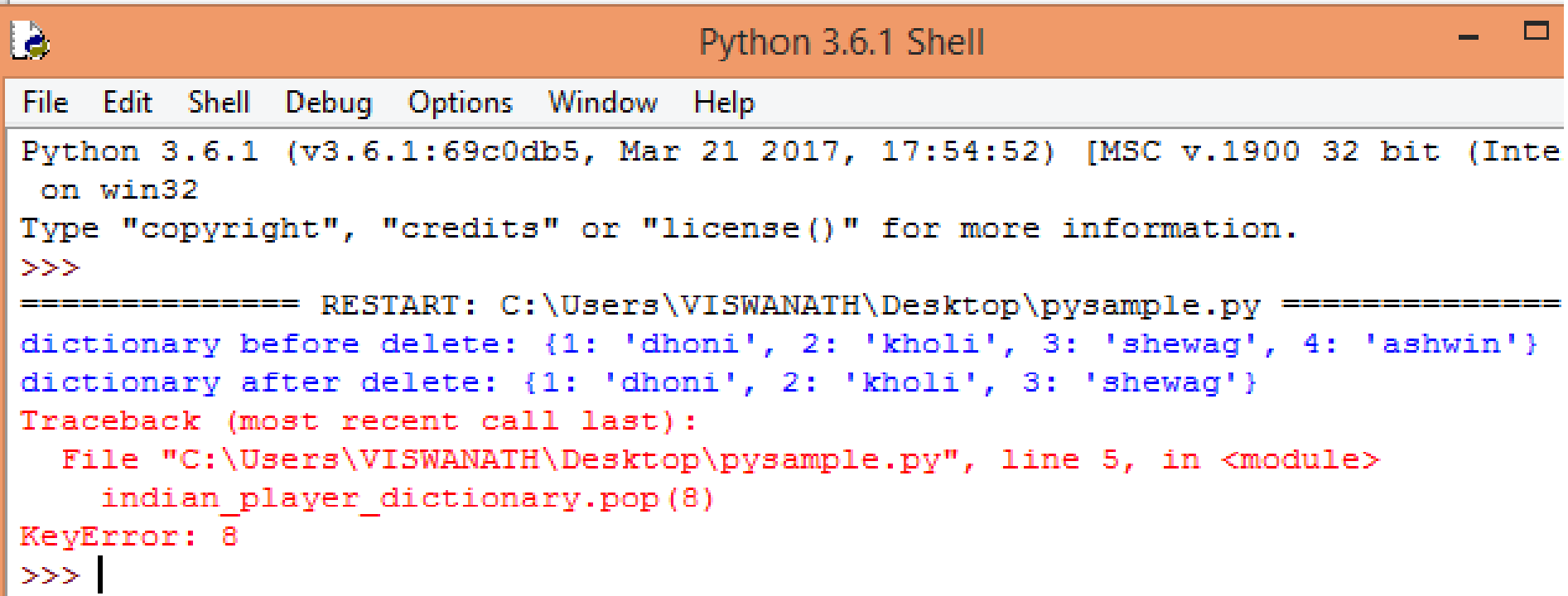
dictionary before delete: {1: 'dhoni', 2: 'kholi', 3: 'shewag', 4: 'ashwin'}

dictionary after delete: {1: 'dhoni', 2: 'kholi', 3: 'shewag'}

>>> |

- **Method2:**Delete the dictionary element by pop method.
- **Syntax:** dictionary_name.pop(key)
- If we try to delete or pop the key that is not present in the dictionary then it will render a key error.

```
indian_player_dictionary={1:"dhoni",2:"kholi",3:"shewag",4:"ashwin"}
print("dictionary before delete:",indian_player_dictionary)
indian_player_dictionary.pop(4)
print("dictionary after delete:",indian_player_dictionary)
indian_player_dictionary.pop(8)
```



The image shows a screenshot of a Python 3.6.1 Shell window. The window has a title bar with the text "Python 3.6.1 Shell" and standard window controls. Below the title bar is a menu bar with options: File, Edit, Shell, Debug, Options, Window, and Help. The main area of the window displays the output of a Python script. The script defines a dictionary named 'indian_player_dictionary' with four key-value pairs. It then prints the dictionary before and after deleting the element with key 4. Finally, it attempts to delete an element with key 8, which results in a 'KeyError: 8'.

```
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit (Intel) on win32]
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =====
dictionary before delete: {1: 'dhoni', 2: 'kholi', 3: 'shewag', 4: 'ashwin'}
dictionary after delete: {1: 'dhoni', 2: 'kholi', 3: 'shewag'}
Traceback (most recent call last):
  File "C:\Users\VISWANATH\Desktop\pysample.py", line 5, in <module>
    indian_player_dictionary.pop(8)
KeyError: 8
>>> |
```

- **Updating / modifying existing element in dictionary:** We can update element (key: value) to dictionary using assignment operator as follows.
- **Syntax:** dictionary_name [new_key_value]=value

```
indian_player_dictionary={1:"dhoni",2:"kholi",3:"shewag",4:"ashwin"}
print("dictionary before adding new element:",indian_player_dictionary)
indian_player_dictionary[5] = "ganguly"
print("dictionary after adding new element:",indian_player_dictionary)
```

Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit (Intel)] on win32

Type "copyright", "credits" or "license()" for more information.

>>>

===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =====

dictionary before adding new element: {1: 'dhoni', 2: 'kholi', 3: 'shewag', 4: 'ashwin'}

dictionary after adding new element: {1: 'dhoni', 2: 'kholi', 3: 'shewag', 4: 'ashwin', 5: 'ganguly'}

>>> |

Use of IN and NOT IN operator (or) check for existence of a key:

Syntax: <key> in <dictionary_name>

<key> not in <dictionary_name>

The IN operator will return True if the key is present in dictionary else will return False.

The NOT IN operator will return true if the given key is not present in the dictionary, else it returns false

```
emp= {"salary":10000,"age":25,"name":"John"}
print("full dict is:",emp)
print("is age is a key?", "age" in emp)
print("is john is a key?", "john" in emp)
print("is python is not key?", "python" not in emp)
```



Python 3.6.1 Shell

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v
on win32

Type "copyright", "credits" or "license()" for more informa

>>>

===== RESTART: C:\Users\VISWANATH\Desktop\pysample

full dict is: {'salary': 10000, 'age': 25, 'name': 'John'}

is age is a key? True

is john is a key? False

is python is not key? True

>>> |

DICTIONARY FUNCTIONS AND METHODS

Python dictionary mainly has 8 inbuilt functions & methods. They are:

- `len()`
- `clear()`
- `get()`
- `has_key()`
- `items()`
- `keys()`
- `values()`
- `cmp()`

len() method: This method returns the length of the dictionary i. e., count of (key: value pairs).

Syntax: len (dictionary_name)

clear() method: This method removes all items from the dictionary and the dictionary becomes empty.

Syntax: dictionary_name.clear()

get() method: With this method you can get the item with given key add to dictionary. If key is not present then “None” will be displayed instead of throwing an error.

Syntax: dictionary_name.get (key)

Example: emp={"name": "John", "salary": 10000, "age": 25}

emp_name=emp.get("name")

print(emp_name)

Output: John

has_key() method: This method checks for the presence of given key in the dictionary

Syntax: dict_name.has_key(key)

Output: Returns true if key is present in the dictionary else it returns false.

items() method: This method returns all the items in the dictionary or sequence of (key,value) in any order or list.

Syntax : dict_name.items()

keys() method: This method returns all the keys in dictionary.

Syntax: dict_name.keys()

values() method: This method returns all the values from the dictionary as a sequence.

Syntax: dict_name.values()

cmp() method: This method is used to compare the dictionaries based on their key values. The answer will be 0 or -1 or +1

Return zero(0) if both dictionaries are equal Return -1 if dict1 < dict2

Return +1 if dict1 > dict2 Syntax:

Cmp(dictionary1, dictionary2)

```
File Edit Format Run Options Window Help
emp= {"salary":10000,"age":25,"name":"John"}
print("dict elements are:",emp)

print("length of dict :",len(emp))

print("GET METHOD FOR NAME:",emp.get("name"))

print("DICT ALL ITEMS :",emp.items())

print("DICT ALL KEYS :",emp.keys())

print("DICT ALL VALUES :",emp.values())

emp.clear()

print("dict elements after clear method:",emp)
|
print("HAS_KEY METHOD FOR AGE :",emp.has_key("age"))
print("HAS_KEY METHOD FOR MOBILE :",emp.has_key("mobile"))
```

File Edit Shell Debug Options Window Help

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit (Intel)]
on win32

Type "copyright", "credits" or "license()" for more information.

>>>

===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =====

dict elements are: {'salary': 10000, 'age': 25, 'name': 'John'}

length of dict : 3

GET METHOD FOR NAME: John

DICTIONARY ALL ITEMS : dict_items([('salary', 10000), ('age', 25), ('name', 'John')])

DICTIONARY ALL KEYS : dict_keys(['salary', 'age', 'name'])

DICTIONARY ALL VALUES : dict_values([10000, 25, 'John'])

dict elements after clear method: {}

SET

- Sets are used to store multiple items in a single variable.
- Set is one of 4 built-in data types in Python used to store collections of data, the other 3 are [List](#), [Tuple](#), and [Dictionary](#), all with different qualities and usage.
- A set is a collection which is *unordered*, *unchangeable**, and *unindexed*.
- Create a Set:

```
thisset = {"apple", "banana", "cherry"}  
print(thisset)
```

Set Items - Data Types

Set items can be of any data type:

String, int and boolean data types:

```
set1 = {"apple", "banana", "cherry"}  
set2 = {1, 5, 7, 9, 3}
```

```
set3 = {True, False, False}
```

Python Set Operations(union,intersection,difference and symmetric difference)

```
A = {0, 2, 4, 6, 8};  
B = {1, 2, 3, 4, 5};  
  
# union  
print("Union :", A | B)  
  
# intersection  
print("Intersection :", A & B)  
  
# difference  
print("Difference :", A - B)
```

```
# symmetric difference  
print("Symmetric difference :", A ^ B)
```

Output:

```
'Union :', set([0, 1, 2, 3, 4, 5, 6, 8]))  
( 'Intersection :', set([2, 4]))  
( 'Difference :', set([8, 0, 6]))  
( 'Symmetric difference :', set([0, 1, 3, 5, 6, 8]))
```

Set (continued)

- A set is a collection which is unordered and unindexed.
- In Python sets are written with curly brackets.
- #Note: the set list is unordered, meaning:the items will appear in a random order.

Example

Create a Set:

```
thisset={"apple","banana","cherry"}  
print(thisset)
```

Output

```
{'cherry','banana','apple'}
```

Access Items

You cannot access items in a set by referring to an index, since sets are unordered the items has no index.

But you can loop through the set items using a `for` loop, or ask if a specified value is present in a set, by using the `in` keyword.

Example

Loop through the set, and print the values:

```
thisset = {"apple", "banana", "cherry"}  
  
for x in thisset:  
    print(x)
```

- Output
apple
banana
cherry

Example

Check if "banana" is present in the set:

```
thisset = {"apple", "banana", "cherry"}  
  
print("banana" in thisset)
```

- Output

True

Change Items

Once a set is created, you cannot change its items, but you can add new items.

Add Items

To add one item to a set use the `add()` method.

To add more than one item to a set use the `update()` method.

Example

Add an item to a set, using the `add()` method:

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.add("orange")  
  
print(thisset)
```

- Output

{'orange','apple','banana','cherry'}

Example

Add multiple items to a set, using the `update()` method:

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.update(["orange", "mango", "grapes"])  
  
print(thisset)
```

- Output

```
{'apple', 'grapes', 'mango', 'banana',  
'cherry', 'orange'}
```

Get the Length of a Set

To determine how many items a set has, use the `len()` method.

Example

Get the number of items in a set:

```
thisset = {"apple", "banana", "cherry"}  
  
print(len(thisset))
```

- Output
3

Remove Item

To remove an item in a set, use the `remove()`, or the `discard()` method.

Example

Remove "banana" by using the `remove()` method:

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.remove("banana")  
  
print(thisset)
```

Note: If the item to remove does not exist, `remove()` will raise an error.

- Output
{'cherry','apple'}

Example

Remove "banana" by using the `discard()` method:

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.discard("banana")  
  
print(thisset)
```

Note: If the item to remove does not exist, `discard()` will **NOT** raise an error.

- Output
{'cherry', 'apple'}

You can also use the `pop()` method to remove an item, but this method will remove the *last* item. Remember that sets are unordered, so you will not know what item that gets removed.

The return value of the `pop()` method is the removed item.

Note: Sets are *unordered*, so when using the `pop()` method, you will not know which item that gets removed.

Example

Remove the last item by using the `pop()` method:

```
thisset = {"apple", "banana", "cherry"}  
  
x = thisset.pop()  
  
print(x)  
  
print(thisset)
```

- Output
banana
{'apple','cherry'}

Example

The `clear()` method empties the set:

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.clear()  
  
print(thisset)
```

- Output
set()

Example

The `del` keyword will delete the set completely:

```
thisset = {"apple", "banana", "cherry"}  
  
del thisset  
  
print(thisset)
```

- **Output**

Traceback(most recent call last):

File "demo_set_del.py", line 5, in <module>

print(thisset)#this will raise an error because the set no
Longer exists

NameError:name'thisset' is not defined

Sets in Python: (Updated content)

- A Set in Python is used to store a collection of items with the following properties.
- No duplicate elements. If try to insert the same item again, it overwrites previous one.
- An unordered collection. When we access all items, they are accessed without any specific order and we cannot access items using indexes as we do in lists.
- Internally use hashing that makes set efficient for search, insert and delete operations. It gives a major advantage over a list for problems with these operations.
- Mutable, meaning we can add or remove elements after their creation, the individual elements within the set cannot be changed directly.

Example of Python Sets:

```
s = {10, 50, 20}
print(s)
print(type(s))
```

Output

```
{10, 50, 20}
<class 'set'>
```

Note : There is no specific order for set elements to be printed

Type Casting with Python Set method

The Python set() method is used for type casting.

```
# typecasting list to set
s = set(["a", "b", "c"])
print(s)
```

```
# Adding element to the set
s.add("d")
print(s)
```

Output

```
{'c', 'b', 'a'}
{'d', 'c', 'b', 'a'}
```

Check unique and Immutable with Python Set

Python sets cannot have duplicate values. While you cannot modify the individual elements directly, you can still add or remove elements from the set.

```
# Python program to demonstrate that
# a set cannot have duplicate values
# and we cannot change its items
```

```
# a set cannot have duplicate values
s = {"Geeks", "for", "Geeks"}
print(s)
```

```
# values of a set cannot be changed
s[1] = "Hello"
print(s)
```

Output:

The first code explains that the set cannot have a duplicate value. Every item in it is a unique value.

The second code generates an error because we cannot assign or change a value once the set is created. We can only add or delete items in the set.

```
{'Geeks', 'for'}
TypeError: 'set' object does not support item assignment
```

Heterogeneous Element with Python Set

Python sets can store heterogeneous elements in it, i.e., a set can store a mixture of string, integer, boolean, etc datatypes.

```
# Python example demonstrate that a set
# can store heterogeneous elements
s = {"Geeks", "for", 10, 52.7, True}
print(s)
```

Output

```
{True, 'for', 'Geeks', 10, 52.7}
```

Python Frozen Sets

Frozen sets in Python are immutable objects that only support methods and operators that produce a result without affecting the frozen set or sets to which they are applied. It can be done with frozenset() method in Python.

While elements of a set can be modified at any time, elements of the frozen set remain the same after creation.

If no parameters are passed, it returns an empty frozenset.

```
# Python program to demonstrate differences
# between normal and frozen set
```

```
# Same as {"a", "b", "c"}
s = set(["a", "b", "c"])
```

```
print("Normal Set")
print(s)
```

```
# A frozen set
fs = frozenset(["e", "f", "g"])
print("\nFrozen Set")
print(fs)
```

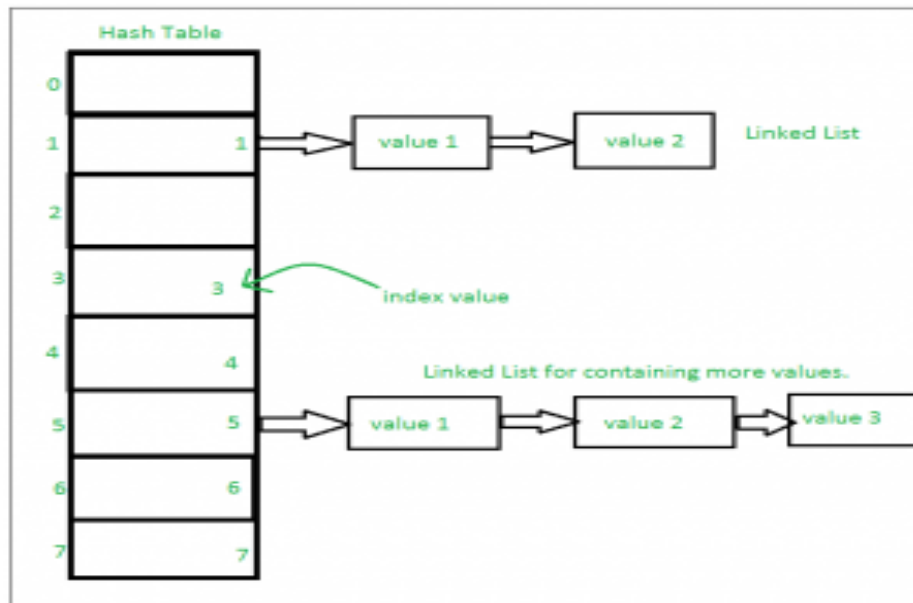
```
# Uncommenting below line would cause error as
# we are trying to add element to a frozen set
# fs.add("h")
```

Internal working of Set

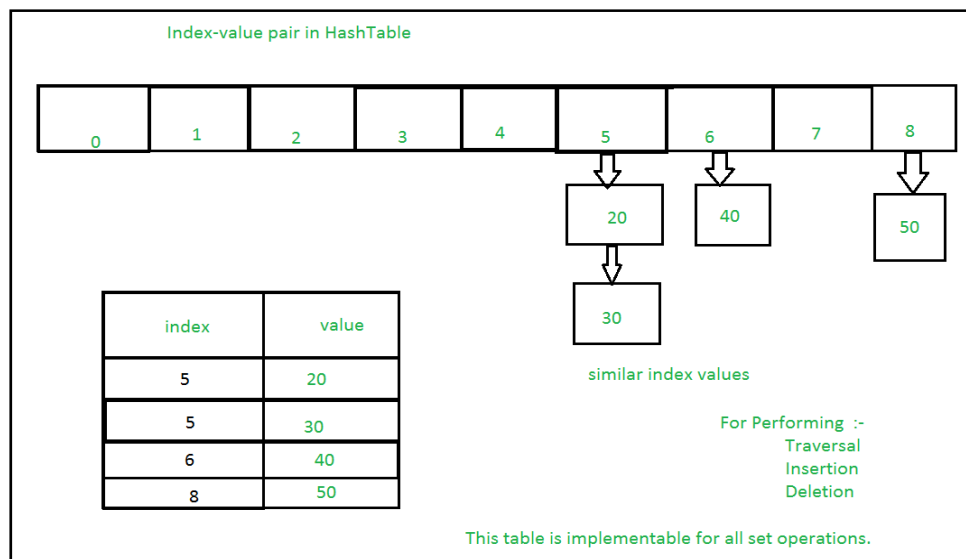
This is based on a data structure known as a hash table. If Multiple values are present at the same index position, then the value is appended to that index position, to form a Linked List.

In, Python Sets are implemented using a dictionary with dummy variables, where key beings the members set with greater optimizations to the time complexity.

Set Implementation:



Sets with Numerous operations on a single HashTable:



Methods for Sets:

Adding elements to Python Sets

Insertion in the set is done through the set.add() function, where an appropriate record value is created to store in the hash table.

```
# A Python program to
# demonstrate adding elements
# in a set

# Creating a Set
people = {"Jay", "Idrish", "Archi"}

print("People:", end = " ")
print(people)

# This will add Daxit
# in the set
people.add("Daxit")

# Adding elements to the
# set using iterator
for i in range(1, 6):
    people.add(i)

print("\nSet after adding element:", end = " ")
print(people)
```

Output

People: {'Idrish', 'Archi', 'Jay'}

Set after adding element: {1, 2, 3, 4, 5, 'Daxit', 'Archi', 'Jay', 'Idrish'}

Union operation on Python Sets

Two sets can be merged using union() function or | operator. Both Hash Table values are accessed and traversed with merge operation perform on them to combine the elements, at the same time duplicates are removed.

```
# Python Program to
# demonstrate union of
# two sets

people = {"Jay", "Idrish", "Archil"}
vampires = {"Karan", "Arjun"}
dracula = {"Deepanshu", "Raju"}

# Union using union()
# function
population = people.union(vampires)
```

```
print("Union using union() function")
print(population)
```

```
# Union using "|"
# operator
population = people|dracula
```

```
print("\nUnion using '|' operator")
print(population)
```

Output

```
Union using union() function
{'Idrish', 'Arjun', 'Jay', 'Karan', 'Archil'}
```

```
Union using '|' operator
{'Idrish', 'Deepanshu', 'Raju', 'Jay', 'Archil'}
```

Intersection operation on Python Sets

This can be done through intersection() or & operator. Common Elements are selected. They are similar to iteration over the Hash lists and combining the same values on both the Table.

```
# Python program to
# demonstrate intersection
# of two sets
```

```
set1 = set()
set2 = set()
```

```
for i in range(5):
    set1.add(i)
```

```
for i in range(3,9):
    set2.add(i)
```

```
# Intersection using
# intersection() function
set3 = set1.intersection(set2)
```

```
print("Intersection using intersection() function")
print(set3)
```

```
# Intersection using
# "&" operator
set3 = set1 & set2
print("\nIntersection using '&' operator")
print(set3)
```

Output

Intersection using intersection() function
{3, 4}

Intersection using '&' operator
{3, 4}

Finding Differences of Sets in Python

To find differences between sets. Similar to finding differences in the linked list. This is done through difference() or – operator.

**# Python program to
demonstrate difference
of two sets**

```
set1 = set()
set2 = set()

for i in range(5):
    set1.add(i)

for i in range(3,9):
    set2.add(i)

# Difference of two sets
# using difference() function
set3 = set1.difference(set2)

print(" Difference of two sets using difference() function")
print(set3)

# Difference of two sets
# using '-' operator
set3 = set1 - set2

print("\nDifference of two sets using '-' operator")
print(set3)
```

Output

Difference of two sets using difference() function
{0, 1, 2}

Difference of two sets using '-' operator
{0, 1, 2}

Clearing Python Sets

Set Clear() method empties the whole set inplace.

**# Python program to
demonstrate clearing
of set**

```
set1 = {1,2,3,4,5,6}
```

```
print("Initial set")
```

```
print(set1)
```

```
# This method will remove
```

```
# all the elements of the set
```

```
set1.clear()
```

```
print("\nSet after using clear() function")
```

```
print(set1)
```

Output

Initial set

{1, 2, 3, 4, 5, 6}

Set after using clear() function

set()

However, there are two major pitfalls in Python sets:

- The set doesn't maintain elements in any particular order.
- Only instances of immutable types can be added to a Python set.

Operators for Sets:

Sets and frozen sets support the following operators:

Operators	Notes
key in s	containment check
key not in s	non-containment check
s1 == s2	s1 is equivalent to s2
s1 != s2	s1 is not equivalent to s2
s1 <= s2	s1 is subset of s2
s1 < s2	s1 is proper subset of s2
s1 >= s2	s1 is superset of s2
s1 > s2	s1 is proper superset of s2

Operators	Notes
$s1 \mid s2$	the union of $s1$ and $s2$
$s1 \ \& \ s2$	the intersection of $s1$ and $s2$
$s1 - s2$	the set of elements in $s1$ but not $s2$
$s1 \wedge s2$	the set of elements in precisely one of $s1$ or $s2$

Advanced list processing:

List Comprehension:

- List comprehensions provide a concise way to apply operations on a list.
- It creates a new list in which each element is the result of applying a given operation in a list.
- It consists of brackets containing an expression followed by a “for” clause, then a list.
- The list comprehension always returns a result list.

Syntax

list=[expression for item in list if conditional]

List Comprehension	Output
>>>L=[x**2 for x in range(0,5)] >>>print(L)	[0, 1, 4, 9, 16]
>>>[x for x in range(1,10) if x%2==0]	[2, 4, 6, 8]
>>>[x for x in 'Python Programming' if x in ['a','e','i','o','u']]	['o', 'o', 'a', 'i']
>>>mixed=[1,2,"a",3,4.2] >>> [x**2 for x in mixed if type(x)==int]	[1, 4, 9]
>>>[x+3 for x in [1,2,3]]	[4, 5, 6]
>>> [x*x for x in range(5)]	[0, 1, 4, 9, 16]
>>> num=[-1,2,-3,4,-5,6,-7] >>> [x for x in num if x>=0]	[2, 4, 6]
>>> str=["this","is","an","example"] >>> element=[word[0] for word in str] >>> print(element)	['t', 'i', 'a', 'e']